

ASTRA

API Security Threat & Risk Atlas

White Paper · Version 1.0 · April 2026

Isha Singh Malik

linkedin.com/in/ishasinghmalik · x.com/ishamalik2427

github.com/isha-singhMalik/Astra

Abstract

ASTRA (API Security Threat & Risk Atlas) is a free, open-source threat matrix for API security, structured after the MITRE ATT&CK framework but built natively for the protocol-specific reality of modern APIs. This white paper presents the ASTRA v1.0 framework: its methodology, threat taxonomy, composite severity scoring system, Sigma detection rule corpus, and five detailed real-world breach case studies. ASTRA addresses a critical gap — no existing resource provides a structured, protocol-native, detection-ready knowledge base of API attack techniques mapped to confirmed public incidents. ASTRA fills that gap and provides a living, community-maintained standard for the API security community.

Keywords: API security, threat matrix, MITRE ATT&CK, OWASP, Sigma rules, BOLA, BFLA, GraphQL security, REST API, detection engineering, threat modelling, breach analysis

1. Introduction

The proliferation of APIs as the backbone of modern digital infrastructure has created an asymmetric security challenge. Organisations expose hundreds or thousands of API endpoints — each a potential attack surface — while the threat intelligence ecosystem has lagged behind in providing structured, protocol-native guidance for understanding, detecting, and remediating API-specific attacks.

Two resources dominate the current landscape. MITRE ATT&CK provides a comprehensive enterprise threat matrix, but its breadth precludes API protocol specificity — it has no concept of GraphQL versus REST versus gRPC, and API attacks are treated as subcategories of broader web application threats. The OWASP API Security Top 10, while API-specific, is a ranked vulnerability list rather than a structured matrix. It does not bundle detection rules, does not provide protocol-specific technique variants, and has not been updated since 2023.

There is no structured, protocol-native, detection-ready threat matrix for API security. ASTRA is built to be exactly that.

ASTRA — API Security Threat & Risk Atlas — addresses this gap. It is a structured threat matrix that organises API attack techniques by adversary tactic, maps each technique to the protocols on which it applies, assigns a composite severity score, bundles a ready-to-use Sigma detection rule, and maps every technique to at least one confirmed public breach.

ASTRA is entirely open source under Creative Commons Attribution 4.0, community-maintained via GitHub pull requests, and versioned semantically — enabling the community to build on a stable, citable reference standard that evolves with the threat landscape.

2. Problem Statement

2.1 The API Attack Surface is Growing Faster Than Defences

API traffic now constitutes the majority of internet traffic. Salt Security's 2024 State of API Security Report found that 94% of organisations experienced API security problems in production in the past year, and that API attack traffic grew by 400% in the same period. Yet most security operations

teams lack a structured taxonomy for API-specific threats — making systematic detection, triage, and remediation significantly harder than for other attack surfaces.

2.2 Existing Resources are Insufficient

Capability	MITRE ATT&CK	OWASP API Top 10	ASTRA
API-specific coverage	Partial	Yes	Yes
Protocol-native tagging (REST/GraphQL/gRPC)	No	No	Yes
Bundled detection rules	No	No	Yes (Sigma)
Composite severity scoring	Partial	Rank only	Yes
Real breach mapping per technique	Some	Some	Yes — every technique
Community PR contribution model	No	No	Yes
Continuous versioned updates	Annual	Periodic	Yes — per PR

2.3 The Detection Gap

Perhaps the most operationally significant gap is detection. Security engineers building SIEM rules for API attacks must source detection logic from scattered blog posts, vendor threat reports, and individual CVE analyses. No existing framework bundles detection rules — in any format — alongside technique descriptions. ASTRA ships a Sigma rule with every technique, enabling immediate operationalisation of threat intelligence into detection pipelines.

3. ASTRA Methodology

3.1 Framework Design Principles

ASTRA is designed around four core principles:

- Protocol-native: every technique specifies the API protocols to which it applies, enabling practitioners to scope their assessment to the protocols in use in their environment.
- Detection-first: every technique ships with a Sigma detection rule, prioritising operational utility over theoretical completeness.
- Evidence-based: every technique maps to at least one publicly confirmed breach, grounding the framework in demonstrated real-world risk rather than theoretical attack paths.
- Community-maintained: ASTRA evolves through GitHub pull requests, with automated validation enforcing quality standards on every contribution.

3.2 Technique Selection Criteria

Techniques are included in ASTRA when they meet all three of the following criteria:

- The technique is specific to API attack surfaces — it exploits properties of API communication protocols, authentication mechanisms, or data exposure patterns rather than generic web application vulnerabilities.
- The technique has been demonstrated in at least one publicly disclosed real-world incident, confirmed security research, or assigned CVE.
- A meaningful detection signal exists — the technique produces observable patterns in API gateway logs, SIEM telemetry, or network traffic that can be captured in a Sigma rule.

3.3 Severity Scoring System

ASTRA uses a composite severity scoring system across four dimensions, each scored 1–5:

Dimension	Definition	Score range	Example anchor (score 5)
Exploitability	Ease of execution by a low-skilled attacker with minimal access	1–5	Unauthenticated, trivial one-line exploit
Prevalence	Frequency of occurrence across real-world API deployments	1–5	Present in the majority of APIs with this feature
Data Sensitivity	Sensitivity of data exposed or operations enabled by the attack	1–5	Critical credentials, financial data, health records
Business Impact	Financial, regulatory, and reputational consequence of exploitation	1–5	Catastrophic — regulatory penalty + mass breach + litigation

The composite score is the arithmetic mean of all four dimensions, rounded to one decimal place.
Rating thresholds: Critical ≥ 4.0 · High ≥ 3.0 · Medium ≥ 2.0 · Low < 2.0.

3.4 Tactic Taxonomy

ASTRA organises techniques into five tactic categories representing the adversary's objective at each stage of an API attack chain:

Tactic	Adversary objective	Techniques (v1.0)
Reconnaissance	Map the API surface: discover endpoints, schemas, and authentication requirements	ASTRA-REC-001 to 003
Authentication Abuse	Circumvent, bypass, or forge authentication mechanisms and tokens	ASTRA-AUTH-001 to 003
Authorization Failure	Access resources or functions beyond the authenticated user's permissions	ASTRA-AUTHZ-001 to 003
Exfiltration	Extract data at scale through API response mechanisms	ASTRA-EXFIL-001 to 002
Impact	Disrupt availability, corrupt data, or abuse business logic for gain	ASTRA-IMPACT-001 to 003

4. ASTRA v1.0 Threat Matrix

The following table presents the complete ASTRA v1.0 matrix. Each technique is documented in full in the repository at github.com/isha-singhMalik/Astra.

ID	Technique	Tactic	Protocols	Severity
ASTRA-REC-001	API endpoint enumeration via JS scraping	Reconnaissance	REST, GraphQL	Medium
ASTRA-REC-002	OpenAPI / Swagger specification discovery	Reconnaissance	REST	High
ASTRA-REC-003	GraphQL introspection leak	Reconnaissance	GraphQL	High
ASTRA-AUTH-001	JWT none algorithm bypass	Auth Abuse	REST, GraphQL	Critical

ASTRA-AUTH-002	OAuth token leakage via referrer header	Auth Abuse	REST	High
ASTRA-AUTH-003	API key stuffing and brute force	Auth Abuse	REST, gRPC	High
ASTRA-AUTHZ-001	Broken Object Level Authorization (BOLA/IDOR)	AuthZ Failure	REST, GraphQL, WS	Critical
ASTRA-AUTHZ-002	Broken Function Level Authorization (BFLA)	AuthZ Failure	REST, GraphQL	Critical
ASTRA-AUTHZ-003	Mass assignment / property injection	AuthZ Failure	REST, GraphQL	High
ASTRA-EXFIL-001	Excessive data exposure	Exfiltration	REST, GraphQL	Critical
ASTRA-EXFIL-002	Bulk data harvesting via pagination abuse	Exfiltration	REST	High
ASTRA-IMPACT-001	Resource exhaustion / API denial of service	Impact	REST, gRPC, WS	High
ASTRA-IMPACT-002	SQL and NoSQL injection via API parameters	Impact	REST, GraphQL	Critical
ASTRA-IMPACT-003	Business logic abuse	Impact	REST, GraphQL	High

5. Breach Case Studies

This section maps five confirmed, publicly disclosed API security breaches to specific ASTRA techniques, demonstrating the real-world applicability of the framework. These are not hypothetical attack scenarios — they are documented incidents with confirmed root causes.

Case Study 1 — Optus Australia (2022) | ASTRA-EXFIL-001

Organisation	Optus (Singtel subsidiary), Australia
Year	2022
Records exposed	Approximately 10 million customer records
Data compromised	Full names, dates of birth, phone numbers, email addresses, passport numbers, driver licence numbers
ASTRA technique	ASTRA-EXFIL-001 — Excessive Data Exposure

ASTRA severity

Critical (composite score: 4.5/5)

What happened

An API endpoint responsible for customer data retrieval was accessible without authentication. The endpoint returned complete customer objects — including identity document numbers — rather than a response filtered to the requesting client's needs. An attacker scripted sequential requests across the customer ID space and exfiltrated the full dataset.

ASTRA-EXFIL-001 mapping

This is a textbook instance of Excessive Data Exposure: the API returned the complete backend model object, relying on the client to filter display. This development pattern — returning full ORM objects for convenience — is the root cause of a disproportionate number of mass data breaches. The filtering logic that existed was in the mobile app frontend, not in the API response.

Detection with ASTRA

The ASTRA-EXFIL-001 Sigma rule flags API responses containing sensitive field names (passport_number, licence_number, tax_id) in JSON keys — a response-side DLP pattern that most organisations apply to email but not to API traffic. Had this rule been deployed at Optus's API gateway, the data exfiltration would have generated alerts from the first request.

```
# ASTRA-EXFIL-001 detection signal
# Flag: API responses containing identity document field names in JSON keys
# Platform: API gateway response inspection / WAF

detection:
  selection_sensitive_fields:
    sc-response-body|contains:
      - 'passport_number'
      - 'licence_number'
      - 'tax_id'
      - 'drivers_licence'
  condition: selection_sensitive_fields
level: high
```

Case Study 2 — Twitter / X (2022) | ASTRA-AUTHZ-001

Organisation	Twitter / X Corp
Year	2022
Records exposed	5.4 million user accounts
Data compromised	Phone numbers and email addresses mapped to account identities
ASTRA technique	ASTRA-AUTHZ-001 — Broken Object Level Authorization (BOLA)
ASTRA severity	Critical (composite score: 4.5/5)

What happened

A Twitter API endpoint designed to check whether a phone number or email address was associated with a Twitter account failed to verify that the requester was the owner of the supplied phone number or email. Any authenticated user could supply any phone number or email and receive back the associated account identity. An attacker enumerated numbers and email addresses at scale, building a database of 5.4 million identity-to-account mappings.

ASTRA-AUTHZ-001 mapping

This is Broken Object Level Authorization — the #1 OWASP API Security risk for two consecutive editions. The API authenticated the requester (confirmed they had a valid session) but failed to authorise the specific object access (confirmed they owned the phone number being queried). This single missing server-side check is the root cause of the breach.

Detection with ASTRA

ASTRA-AUTHZ-001 detection focuses on enumeration velocity: a single authenticated session accessing more than N distinct object identifiers within a short time window. For the Twitter breach pattern specifically, the signal is a single token querying sequential phone number ranges at high frequency — distinguishable from legitimate use by rate and sequence pattern.

Case Study 3 — Peloton (2021) | ASTRA-AUTHZ-001 + ASTRA-REC-002

Organisation	Peloton Interactive
Year	2021
Records exposed	All user profiles — millions of accounts

Data compromised	Age, weight, workout history, location data, gender, private account contents
ASTRA techniques	ASTRA-AUTHZ-001 (BOLA) + ASTRA-REC-002 (API spec discovery)

What happened

Researchers discovered that Peloton's API returned full user profile data — including profiles explicitly set to private — for any user ID, with no authentication required. The API specification was also discoverable via standard spec paths, providing researchers (and potential attackers) with a complete endpoint inventory. Peloton took 90 days to remediate after initial responsible disclosure, during which the vulnerability remained exploitable.

ASTRA dual-technique mapping

This breach illustrates a common attack chain: ASTRA-REC-002 (OpenAPI spec discovery) enabling ASTRA-AUTHZ-001 (BOLA). The attacker first mapped the full API surface using the exposed specification, then exploited the missing authorization check on user object endpoints. Neither vulnerability alone is as dangerous as the combination — the spec accelerates enumeration by eliminating the guesswork of endpoint discovery.

Case Study 4 — Auth0 / jsonwebtoken (2015) | ASTRA-AUTH-001

Organisation	Auth0 and downstream library consumers
Year	2015 (CVE-2015-9235)
Impact	Complete authentication bypass for all applications using affected libraries
ASTRA technique	ASTRA-AUTH-001 — JWT None Algorithm Bypass
ASTRA severity	Critical (composite score: 4.25/5)

What happened

Auth0's node-jsonwebtoken library, and numerous other JWT libraries across multiple languages, accepted JSON Web Tokens in which the algorithm field was set to 'none' — indicating an unsigned token requiring no cryptographic verification. An attacker could take any valid JWT, modify the payload to claim any identity or role, set the algorithm to none, strip the signature, and present the resulting token. Libraries that accepted this would treat the forged token as legitimate.

ASTRA-AUTH-001 mapping

This vulnerability illustrates a fundamental design flaw in naive JWT verification: trusting the token's own declaration of how it should be verified. A securely implemented JWT verifier should enforce the expected algorithm server-side, never accepting the client-supplied algorithm field as authoritative. The ASTRA detection rule for this technique has zero expected false positives in production — no legitimate API should ever receive an alg:none JWT.

```
# Forged JWT structure - ASTRA-AUTH-001
# Header (Base64URL decoded): {"alg": "none", "typ": "JWT"}
# Base64URL encoded header: eyJhbGciOiJub25lIiwidHlwIjoisIldUIIn0

# Detection: flag any Authorization header where the JWT header
# decodes to alg:none, alg:None, or alg:NONE

detection:
  selection_none_alg:
    cs-authorization|contains:
      - 'eyJhbGciOiJub25lIi' # {"alg":"none"}
      - 'eyJhbGciOiJOT05FIi' # {"alg":"NONE"}
    condition: selection_none_alg
  level: critical
  falsepositives:
    - None expected in production
```

Case Study 5 — HackerOne GraphQL (2020) | ASTRA-REC-003

Organisation	HackerOne
Year	2020
Impact	Full internal GraphQL schema exposed including references to undisclosed vulnerability reports
ASTRA technique	ASTRA-REC-003 — GraphQL Introspection Leak
ASTRA severity	High (composite score: 3.75/5)

What happened

A researcher discovered that HackerOne's GraphQL API had introspection enabled in production. A single introspection query returned the complete API schema — including internal query types with

field names referencing undisclosed vulnerability reports. The schema itself constituted sensitive information disclosure, and the exposed field structure provided a map for further probing of the API's internal data model.

ASTRA-REC-003 mapping

GraphQL introspection is enabled by default in most GraphQL frameworks and is an essential development tool. The security failure is leaving it enabled in production, where it provides any unauthenticated requester with a complete machine-readable schema of the entire API in a single request. ASTRA-REC-003 documents this technique and provides the detection signal (requests containing `__schema` or `IntrospectionQuery` in the POST body) and the one-line remediation (disable introspection in production via framework configuration).

6. Detection Engineering with ASTRA

6.1 Sigma Rule Corpus

ASTRA ships one Sigma detection rule per technique — 14 rules in v1.0. Sigma is a vendor-neutral, open rule format supported by Splunk, Elastic SIEM, Microsoft Sentinel, IBM QRadar, Sumo Logic, and any other Sigma-compatible backend. ASTRA rules are stored in `detection-rules/sigma/` in the repository and can be converted to any supported backend using `sigma-cli`.

```
# Convert ASTRA Sigma rules to your SIEM
# Install sigma-cli
pip install sigma-cli

# Convert to Splunk SPL
sigma convert -t splunk detection-rules/sigma/ASTRA-AUTH-001.yml

# Convert to Elastic ESQ
sigma convert -t elasticsearch detection-rules/sigma/ASTRA-AUTHZ-001.yml

# Convert all 14 rules at once to Microsoft Sentinel KQL
sigma convert -t microsoft365defender detection-rules/sigma/*.yml
```

6.2 Detection Philosophy

ASTRA detection rules are designed around three principles. First, behavioural specificity: rules target the specific observable behaviour of the attack, not generic anomalies. Second, low false positive tolerance: rules are designed to minimise noise — a detection rule that fires constantly trains engineers to ignore it. Third, layered detection: where a single rule cannot reliably detect a technique without false positives, ASTRA supplements with behavioural indicators that can be combined with threat hunting queries.

6.3 API Gateway vs SIEM Detection

ASTRA rules are written for both deployment contexts. Some techniques are best detected at the API gateway layer — where request and response content is available before it reaches application logs. Others are best detected in a SIEM where cross-session correlation is possible. Each technique page in ASTRA specifies the recommended detection layer and provides a WAF/gateway rule hint in addition to the Sigma rule.

7. Using ASTRA in Practice

7.1 API Threat Modelling

ASTRA provides a structured checklist for API threat modelling sessions. For each endpoint group in scope, practitioners work through each tactic category and assess whether the endpoint is vulnerable to each applicable technique. The protocol tags on each technique enable scoping — a team working on a GraphQL API can immediately filter to GraphQL-applicable techniques.

The recommended threat modelling workflow using ASTRA:

1. Enumerate API surface: list all endpoints, group by function and data sensitivity
2. Identify protocols in use: REST, GraphQL, gRPC, WebSocket, SOAP
3. Filter ASTRA matrix to applicable protocol tags
4. For each technique in scope, assess whether the attack vector exists
5. Prioritise remediation by ASTRA composite severity score
6. Map detection rules to existing SIEM or API gateway

7.2 Penetration Testing

For penetration testers, ASTRA organises the API testing engagement by adversary tactic rather than by vulnerability class. This mirrors real attacker behaviour and ensures systematic coverage. Each technique page includes an attack scenario walkthrough and example request or payload, enabling testers to move efficiently from technique identification to exploitation proof-of-concept.

7.3 Security Operations and Detection Engineering

For security operations teams, ASTRA's Sigma rule corpus provides an immediate starting point for API attack detection. Teams can import all 14 rules into their SIEM, tune thresholds for their environment, and use the behavioural indicators on each technique page to build complementary threat hunting queries. As ASTRA grows, the detection rule corpus grows with it.

8. Roadmap

8.1 ASTRA v1.1 — Planned

- 10 additional techniques across existing tactic categories
- gRPC-specific technique expansion — streaming attack patterns
- WebSocket abuse techniques — connection hijacking, message injection
- CI/CD pipeline API attack techniques
- API gateway misconfiguration techniques
- Improved Sigma rules for all v1.0 techniques based on community feedback

8.2 ASTRA v2.0 — Planned

- New tactic: Persistence — maintaining long-term unauthorised API access
 - New tactic: Supply Chain — third-party API and SDK compromise
 - Interactive filterable web matrix — searchable by protocol, severity, tactic
 - ASTRA scoring API — JSON endpoint for integration with security tooling
 - ASTRA-based threat report generator — upload OpenAPI spec, receive threat model
-

9. Community and Contribution

ASTRA is an open-source, community-maintained project. Contributions are welcomed and encouraged from security researchers, practitioners, detection engineers, and developers. The contribution workflow is designed to be accessible while maintaining quality standards.

Every technique contribution requires: a completed technique file in the ASTRA markdown template, a corresponding Sigma detection rule, at least one real-world breach mapping, and a severity score with dimension-by-dimension rationale. A GitHub Actions validation bot automatically checks all of these requirements on every pull request, rejecting submissions that do not meet the standard.

Contribution channels:

- New technique proposals: GitHub Issues using the New Technique template
- New breach mappings: GitHub Issues using the Breach Mapping template
- Detection rule improvements: GitHub Issues using the Detection Rule template
- Pull requests for all of the above: github.com/isha-singhMalik/Astra/pulls

10. Conclusion

The five breaches documented in this white paper — Optus, Twitter, Peloton, Auth0, and HackerOne — collectively exposed data belonging to tens of millions of people. None of them required sophisticated offensive capabilities. All of them exploited well-understood, named attack patterns that appear in ASTRA's v1.0 matrix.

The security community has the knowledge to prevent these attacks. What has been missing is a structured, accessible, detection-ready reference that organises that knowledge into a form practitioners can act on. ASTRA is built to be that reference — protocol-native, community-maintained, and freely available to every security engineer, pentester, developer, and organisation that needs it.

Every technique documented in ASTRA is an attack that becomes harder to execute undetected. Every detection rule deployed is a breach that becomes more

likely to be caught. The goal is simple: make API attacks harder and make defenders faster.

ASTRA v1.0 is a starting point. The community will make it into a standard.

References

- [1] OWASP API Security Project. OWASP API Security Top 10 — 2023. owasp.org/API-Security/
- [2] MITRE Corporation. MITRE ATT&CK Framework. attack.mitre.org/
- [3] Salt Security. State of API Security Report, 2024. salt.security/
- [4] ITNews Australia. Optus breach — an API with no authentication. itnews.com.au, 2022.
- [5] BleepingComputer. Twitter confirms zero-day used to expose data of 5.4M accounts. 2022.
- [6] Tenable Research. Peloton API security flaw. tenable.com/blog, 2021.
- [7] Auth0. Critical vulnerabilities in JSON Web Token libraries. auth0.com/blog, 2015.
- [8] HackerOne. GraphQL introspection disclosure — report #291531. hackerone.com, 2020.
- [9] SigmaHQ. Sigma — Generic Signature Format for SIEM Systems. github.com/SigmaHQ/sigma
- [10] CVE-2015-9235. NVD — JWT algorithm confusion. nvd.nist.gov/

About the Author

Isha Singh Malik is the creator and lead maintainer of ASTRA — API Security Threat & Risk Atlas. ASTRA was conceived and built in April 2026 in response to the gap between existing API security resources and the operational needs of security practitioners.

Connect and access ASTRA

GitHub repository: github.com/isha-singhMalik/Astra

Live documentation: isha-singhmalik.github.io/Astra

LinkedIn: linkedin.com/in/ishasinghmalik

Twitter / X: x.com/ishamalik2427

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

You may use, share, and adapt ASTRA freely with attribution.

Citation: Singh Malik, I. (2026). ASTRA — API Security Threat & Risk Atlas
White Paper v1.0. github.com/isha-singhMalik/Astra